

Relatório de Projeto

GREENHERB

Plataforma Web de Gestão Inteligente de uma Estufa

REALIZADO POR

Bruno Salvador	Nº 53588
Letizia Coelli	Nº 53640
Júlia Santos	Nº 53754

Licenciatura em Engenharia Informática

Unidade Curricular de Programação Web

Docentes: Carlos Cunha e Nuno Pereira

2025/2026 · Semestre 2

28 de maio de 2026

Resumo

O presente projeto, denominado GREENHERB, foca-se na resolução dos desafios associados à gestão eficiente de estufas agrícolas, especificamente no cultivo de ervas aromáticas. A complexidade do acompanhamento contínuo de lotes de cultivo, a monitorização de condições ambientais e a gestão de tarefas diárias exigem uma solução tecnológica que centralize a informação e facilite a tomada de decisão em tempo real.

Para endereçar este problema, foi desenvolvida uma aplicação web *full-stack*. A abordagem técnica assenta numa arquitetura cliente-servidor. O *backend* consiste numa API RESTful desenvolvida em Node.js e documentada através da especificação OpenAPI, utilizando o MongoDB para a persistência de dados. A segurança e o controlo de acessos são rigorosamente mantidos através de autenticação com *JSON Web Tokens* (JWT) e autorização baseada em papéis de utilizador. No *frontend*, foi criada uma interface dinâmica que tira partido do armazenamento no *browser* (nomeadamente *IndexedDB* e *Service Workers*) para otimizar o desempenho, oferecer resiliência na sincronização de dados e melhorar a experiência de utilização.

Como principais resultados, o projeto entrega uma plataforma plenamente funcional e segura que permite aos administradores e operadores registar planos de cultivo, monitorizar medições de sensores e gerir tarefas de forma centralizada. A solução final demonstra a aplicação bem-sucedida de padrões modernos de desenvolvimento web, garantindo um sistema escalável e robusto para a otimização da produção agrícola.

Palavras-chave: aplicação web, gestão de estufa, Node.js, MongoDB, armazenamento no browser, JWT, OpenAPI.

Conteúdo

Resumo	i
1 Introdução	1
1.1 Enquadramento	1
1.2 Objetivos	1
1.3 Contexto e restrições	2
1.4 Estrutura do documento	2
2 Requisitos	3
2.1 Atores do sistema	3
2.2 Regras de negócio	4
2.3 Requisitos funcionais	4
2.3.1 Detalhe e critérios de aceitação	5
2.4 Requisitos não funcionais	7
3 Arquitetura	8
3.1 Visão geral	8
3.2 Matriz de responsabilidades	8
3.3 Camada de cliente (<i>browser</i>)	9
3.3.1 Mecanismos de armazenamento	10
3.3.2 Decisão sobre o armazenamento do JWT	10
3.3.3 Estratégia de sincronização	11
3.4 Camada de servidor (API)	11
3.4.1 Cadeia de <i>middleware</i> e Rotas	11
3.4.2 Motor de regras IoT	12
3.5 Modelo de dados	12
3.6 Segurança	13
3.7 Resumo das decisões	14
4 Conclusão	16
4.1 Síntese do trabalho realizado	16
4.2 Avaliação dos critérios de aceitação	16
4.3 Reflexão crítica	17
4.3.1 O que correu bem	17
4.3.2 O que faríamos diferente	17
A Especificação OpenAPI	18
B Dados de demonstração	19

C Distribuição do trabalho	20
-----------------------------------	-----------

Lista de Figuras

3.1	Visão de alto nível da arquitetura em três camadas.	8
3.2	Modelo conceptual e associações na base de dados NoSQL.	13

Lista de Tabelas

2.1	Perfis de utilizador e respetivas responsabilidades.	3
2.2	Lista priorizada de requisitos funcionais.	5
2.3	Requisitos não funcionais.	7
3.1	Distribuição de responsabilidades por camada.	9
3.2	Mecanismos de armazenamento no <i>browser</i> e respetivos propósitos.	10
3.3	Síntese das decisões de arquitetura e segurança.	15
4.1	Verificação dos critérios de aceitação do projeto.	17
C.1	Contribuição de cada elemento da equipa.	20

1

Introdução

1.1 Enquadramento

A empresa GREENHERB pretende informatizar a gestão de uma estufa dedicada à produção de ervas aromáticas, por exemplo manjerição, hortelã, alecrim e tomilho. O sistema atual, maioritariamente manual, cria dificuldades no planeamento, na monitorização das condições ambientais e na rastreabilidade das intervenções realizadas.

O impacto destas dificuldades reflete-se diretamente na operação quotidiana da estufa, resultando em falhas de comunicação na atribuição de tarefas agrícolas, atrasos na identificação de anomalias ambientais, como desvios críticos de temperatura ou humidade, e na perda de histórico vital para o controlo de qualidade dos lotes de cultivo. Neste cenário, a digitalização e a centralização de dados representam uma oportunidade estratégica fundamental, permitindo automatizar fluxos de trabalho, assegurar a integridade dos dados e dotar os operadores de ferramentas ágeis para tomadas de decisão rápidas no terreno.

1.2 Objetivos

O presente trabalho tem como objetivos:

1. Desenvolver uma aplicação web *full-stack* que suporte o ciclo de vida completo de lotes de cultivo, incluindo a gestão de planos, registo de medições e controlo de alertas operacionais;
2. Implementar um sistema robusto de autenticação e autorização baseado em perfis (Técnico, Responsável e Administrador), garantindo o controlo de acessos e a rastreabilidade absoluta das ações através de logs de auditoria;
3. Especificar e documentar a API RESTful em conformidade com a norma OpenAPI 3.x, assegurando a validação de dados e uma interface de comunicação limpa;
4. Desenvolver mecanismos de segurança a nível do servidor, incluindo *rate limiting*, sanitização de dados contra injeções e cabeçalhos de segurança HTTP;
5. Definir uma arquitetura de armazenamento no browser com suporte a *Service Workers* e *IndexedDB*, viabilizando o funcionamento resiliente e a sincronização local de dados da aplicação em condições de conectividade instável.

1.3 Contexto e restrições

A solução é desenvolvida com restrições técnicas previamente definidas: o *frontend* em HTML, CSS e JavaScript nativo, o *backend* em Node.js com recurso ao Mongoose para acesso a uma base de dados NoSQL MongoDB, autenticação baseada em JWT e uso obrigatório dos mecanismos de armazenamento e cache do navegador (`localStorage`, `IndexedDB` e `Cache API`).

Acresce o contexto operacional: a aplicação é utilizada num ambiente físico de estufa onde a conectividade de rede móvel ou Wi-Fi pode ser instável ou nula. Esta realidade condicionou fortemente as decisões de arquitetura do lado do cliente, obrigando ao desenvolvimento de uma lógica de sincronização (*sync*) em segundo plano para persistir dados localmente no `IndexedDB` até que a ligação ao servidor central seja restabelecida.

1.4 Estrutura do documento

O restante documento organiza-se da seguinte forma. O Capítulo 2 identifica e especifica os requisitos do sistema, distinguindo entre funcionais e não funcionais e apresentando a sua respetiva priorização. O Capítulo 3 descreve detalhadamente a arquitetura adotada, explicitando as responsabilidades de cada camada (*browser*, API e base de dados), o modelo de dados em MongoDB e justificando as decisões técnicas tomadas. Por fim, o Capítulo 4 sintetiza as conclusões obtidas, traça um balanço crítico face aos objetivos propostos e sugere caminhos para trabalho futuro.

2

Requisitos

Este capítulo identifica, especifica e prioriza os requisitos do sistema GREENHERB. Define-se o comportamento esperado da plataforma através da caracterização dos utilizadores, do estabelecimento das regras de negócio que governam a operação da estufa e da listagem detalhada dos requisitos funcionais e não funcionais, garantindo a total rastreabilidade do sistema desenvolvido.

2.1 Atores do sistema

O sistema considera três perfis de utilizador distintos, cujas permissões e responsabilidades foram mapeadas diretamente nos módulos de controlo de acessos da aplicação web.

Tabela 2.1: Perfis de utilizador e respetivas responsabilidades.

Perfil	Responsabilidades principais
Técnico	Operação direta no terreno. Responsável pelo registo manual de medições ambientais, consulta e execução de tarefas diárias de rega ou fertilização, e visualização dos lotes de cultivo que lhe estão atribuídos.
Responsável	Gestão agrícola e tomada de decisão. Responsável pela criação de planos de cultivo, parametrização de limites de sensores, associação de planos a novos lotes, triagem de alertas operacionais e consulta de relatórios de produtividade.
Administrador	Gestão técnica da plataforma. Responsável pelo ciclo de vida das contas de utilizador (criação, edição e desativação), atribuição de papéis de segurança e consulta exclusiva dos logs de auditoria global do sistema.

2.2 Regras de negócio

As regras de negócio definem as restrições operacionais, lógicas e de segurança que condicionam o comportamento dos algoritmos e das rotas do servidor.

- RN-01** Cada plano de cultivo está obrigatoriamente associado a uma espécie de erva aromática registada e classifica-se estritamente como *regular*, *emergência* ou *pontual*.
- RN-02** Um plano classificado como *regular* exige a definição explícita dos intervalos ideais de temperatura, humidade e luminosidade, além do cronograma de rega, fertilização e a duração estimada do ciclo biológico.
- RN-03** Um plano *pontual* ou de *emergência* constitui uma exceção operacional e requer validação e assinatura digital (ID de utilizador) do Responsável no momento da associação a um lote.
- RN-04** Os alertas do sistema são classificados como *Informativo*, *Aviso* ou *Crítico*. Um alerta ativo só pode ser arquivado sob o estado de *Resolvido* ou *Ignorado*; a transição para *Ignorado* bloqueia a interface até que seja introduzida uma justificação textual com um mínimo de 10 caracteres.
- RN-05** O sistema suporta os modos de operação *Manual* e *Automático*. No modo automático, a API atua como um motor de regras avançado (IoT), avaliando as medições em tempo real e gerando e executando imediatamente tarefas corretivas (ex: ativação de rega ou climatização). No modo manual, o sistema limita-se a sugerir essas ações, gerando as tarefas no estado *Pendente* para intervenção humana.
- RN-06** **(Segurança e Bloqueio)** O sistema aplica *rate limiting* por endereço IP. Se um utilizador falhar a autenticação mais de 5 vezes consecutivas num intervalo de 15 minutos, o IP é temporariamente bloqueado para mitigar ataques de força bruta.
- RN-07** **(Persistência Offline)** Perante a ausência de conectividade à Internet, as operações de registo de medições e conclusão de tarefas efetuadas pelo Técnico não podem ser rejeitadas; devem ser armazenadas localmente no *IndexedDB* do navegador.
- RN-08** **(Imutabilidade de Auditoria)** Qualquer operação de escrita, modificação ou eliminação de dados críticos (utilizadores, planos, lotes e alertas) gera um registo automático e imutável na coleção de logs de auditoria, contendo o carimbo temporal, IP, ID do ator e descrição da ação.

2.3 Requisitos funcionais

Os requisitos funcionais detalham as ações que a aplicação GREENHERB é capaz de executar. A sua priorização foi estabelecida de acordo com a metodologia MoSCoW: **M**

(*must*, obrigatório para o funcionamento básico), **S** (*should*, importante com elevado valor de negócio), **C** (*could*, desejável se houver margem de tempo), **W** (*won't*, planeado para versões futuras).

Tabela 2.2: Lista priorizada de requisitos funcionais.

ID	Descrição	Atores	Prio.
RF-01	Autenticação de utilizadores baseada em JWT e controlo de acessos por papel.	Todos	M
RF-02	Gestão de utilizadores (criar, editar e desativar contas) via painel administrativo.	Admin.	M
RF-03	Registo, edição, consulta e listagem de espécies de ervas aromáticas.	Resp.	M
RF-04	Criação e parametrização de planos de cultivo (limites de sensores e calendarização).	Resp.	M
RF-05	Criação de lotes de cultivo associados a um plano e monitorização do seu estado.	Resp., Téc.	M
RF-06	Listagem de tarefas diárias atribuídas e capacidade de marcação como concluída.	Téc.	M
RF-07	Registo manual ou simulado de medições ambientais associadas a um lote específico.	Téc.	M
RF-08	Monitorização em segundo plano e geração automática de alertas por desvio métrico.	Sistema	M
RF-09	Triagem de alertas ativos (marcar como Resolvido ou Ignorado com texto justificativo).	Resp.	M
RF-10	Registo automatizado de logs de auditoria contendo metadados de segurança.	Sistema	M
RF-11	Mecanismo de sincronização automática (<i>background sync</i>) dos dados retidos offline.	Sistema	M
RF-12	Dashboard interativo com indicadores gráficos do estado da estufa e alertas ativos.	Todos	S
RF-13	Consulta de histórico cronológico de medições e intervenções por lote.	Todos	S
RF-14	Exportação de relatórios analíticos de produção e medições em formato CSV.	Resp., Admin.	S
RF-15	Recuperação de palavra-passe através do envio e validação de <i>tokens</i> temporários.	Todos	S

2.3.1 Detalhe e critérios de aceitação

Para garantir a qualidade do desenvolvimento e a verificabilidade do sistema, detalham-se de seguida os critérios de aceitação para os requisitos fundamentais da aplicação.

RF-01 — Autenticação e controlo de acessos. O sistema deve assegurar que apenas utilizadores credenciados acedem às áreas privadas e que as permissões respeitam estritamente a tabela de atores.

Critérios de aceitação:

- O login bem-sucedido emite um par de chaves/tokens criptográficos armazenados de forma segura no cliente via Cookies (*HttpOnly*), prevenindo vulnerabilidades de injeção XSS.
- Tentativas de acesso direto às rotas protegidas (ex: */admin/**) por parte de um utilizador sem permissões adequadas resultam num desvio para a página de erro/login, acompanhado de código 403 *Forbidden*.

RF-07 — Registo de medições ambientais. O Técnico deve conseguir submeter os dados recolhidos na estufa através de um formulário dedicado, independentemente da estabilidade da rede sem fios.

Critérios de aceitação:

- Estando a aplicação *online*, a submissão persiste os dados diretamente no MongoDB através de um pedido *POST* com resposta imediata do servidor.
- Estando a aplicação *offline*, o script intercetor redireciona o payload de dados para a fila de espera no *IndexedDB*, exibe uma mensagem informativa de sucesso local e agenda a sincronização assíncrona.

RF-08 — Geração automática de alertas. Sempre que uma nova medição ambiental é persistida, o servidor valida os valores cruzando-os com as condições ideais do plano de cultivo em vigor.

Critérios de aceitação:

- O sistema gera alertas com o nível adequado (Aviso ou Crítico) consoante a violação ou falha do sensor reportada na medição.
- A avaliação das medições contempla regras compostas (ex: anomalias de temperatura e humidade em simultâneo) e, se o lote estiver em modo *Automático*, gera de imediato as tarefas de mitigação apropriadas (como 'climatização' ou 'iluminação'), incluindo a descrição contextual da ação efetuada.

RF-11 — Sincronização automática offline. A aplicação deve gerir autonomamente a transição de estados de conectividade da rede móvel ou Wi-Fi na estufa.

Critérios de aceitação:

- No instante em que o navegador aciona o evento *online*, a arquitetura de sincronização dispara o processo de esvaziamento das filas pendentes no *IndexedDB*, efetuando pedidos recursivos para a API REST.

- Após a confirmação de receção pelo servidor (201 **Created**), os registos locais temporários são integralmente limpos para evitar duplicação ou incoerência de dados.

2.4 Requisitos não funcionais

Os requisitos não funcionais determinam as qualidades, restrições tecnológicas e atributos de segurança sob os quais o sistema GREENHERB opera.

Tabela 2.3: Requisitos não funcionais.

ID	Categoria	Descrição
RNF-01	Segurança	As credenciais de acesso são cifradas com uma função de <i>hashing</i> unidirecional robusta moderna (<i>Argon2</i>) reforçada com <i>salt</i> antes da gravação na base de dados.
RNF-02	Segurança	Proteção do tráfego através da aplicação obrigatória do protocolo HTTPS e configuração de cabeçalhos de segurança HTTP (<i>Content Security Policy</i> , <i>X-Frame-Options</i>).
RNF-03	Segurança	Sanitização rigorosa de todos os dados de entrada na API REST para impedir vulnerabilidades de <i>NoSQL Injection</i> e <i>Cross-Site Scripting</i> (XSS).
RNF-04	Resiliência	Arquitetura <i>Offline-First</i> com recurso a <i>Service Workers</i> e <i>Cache API</i> para garantir que os ficheiros estáticos (HTML, CSS, JS) carregam mesmo sem rede.
RNF-05	Desempenho	O tempo de resposta do servidor para rotas de leitura de dados não deve exceder os 200 milissegundos sob condições normais de carga operacional.
RNF-06	Usabilidade	Interface desenvolvida em <i>Responsive Web Design</i> , adaptando-se ergonomicamente a ecrãs de computadores (<i>desktop</i>) e dispositivos móveis (<i>smartphones</i>).
RNF-07	Manutenibilidade	Código estruturado segundo o padrão arquitetural MVC, com rotas isoladas e a API inteiramente documentada e validada em conformidade com o padrão OpenAPI 3.x.
RNF-08	Escalabilidade	Utilização de base de dados NoSQL (<i>MongoDB</i>) orientada a documentos, facilitando o crescimento horizontal do volume de leituras dos sensores.

3

Arquitetura

A arquitetura do sistema GREENHERB foi desenhada com o propósito de garantir elevada segurança, desempenho e resiliência, dado o seu contexto de operação numa estufa agrícola onde a conectividade de rede pode oscilar. Este capítulo descreve a estrutura física e lógica da aplicação, justificando as decisões técnicas tomadas na distribuição de responsabilidades pelas várias camadas.

3.1 Visão geral

A solução adota uma arquitetura rigorosa em três camadas *full-stack*, separando a interface (cliente), a lógica de negócio (servidor) e a persistência de dados. A Figura 3.1 ilustra esta visão macro e os protocolos de comunicação adotados.

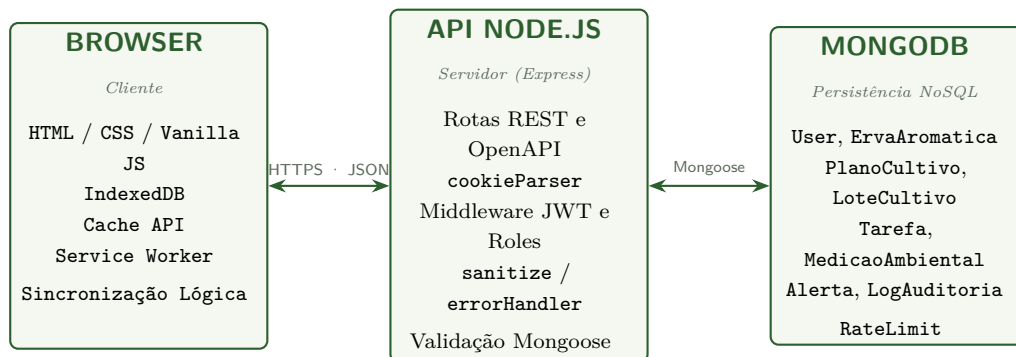


Figura 3.1: Visão de alto nível da arquitetura em três camadas.

A comunicação entre o navegador cliente e o servidor baseia-se em pedidos HTTPS transportando cargas JSON (*payloads*). O servidor Node.js acede e gere a base de dados através do *Object Data Modeling* (ODM) **Mongoose**, assegurando uma abstração que impõe esquemas estritos de validação antes de qualquer persistência.

3.2 Matriz de responsabilidades

A Tabela 3.1 especifica a distribuição de responsabilidades pelos componentes arquiteturais para as operações fulcrais do sistema. Esta clareza de separação impede a sobrecarga computacional do cliente e blindar a lógica sensível no servidor.

Tabela 3.1: Distribuição de responsabilidades por camada.

Funcionalidade	Browser	API	Base de Dados
Autenticação	Recolha de credenciais; redirecionamento com base no <i>script checkSession.js</i> .	Validação e geração de JWT; definição do <i>cookie httpOnly</i> .	Armazenamento seguro do <i>hash</i> da <i>password</i> do <i>User</i> (via Argon2).
Medições Ambientais	Interface de recolha; interação via <i>api.js</i> ; gravação <i>offline</i> em <i>IndexedDB</i> .	Validação cruzada de dados contra os limites definidos no <i>PlanoCultivo</i> .	Gravação atômica na coleção <i>MedicaoAmbiental</i> .
Motor de Regras IoT	Exibição gráfica <i>real-time</i> de alertas e estado na Dashboard.	Avaliação de regras compostas de anomalia; geração de alertas e injeção automática de Tarefas corretivas.	Persistência nas coleções <i>Alerta</i> e <i>Tarefa</i> associadas ao <i>LoteCultivo</i> .
Auditoria Global	Não aplicável (sem controlo ou acesso cliente a este domínio).	Registo transversal via controlador de auditoria para operações tipo POST, PUT, DELETE.	Gravação e retenção inalterável na coleção <i>LogAuditoria</i> .
Sincronização (Sync)	<i>Service Worker</i> monitoriza a ligação e o <i>sync.js</i> retransmite a fila <i>offline</i> .	Receção, validação, verificação de duplicados e emissão de confirmação de receção (201).	Persistência final das operações que ocorreram de modo assíncrono.

3.3 Camada de cliente (*browser*)

O desenvolvimento do cliente abdicou de *frameworks* massivas em prol de uma abordagem puramente baseada em *Vanilla JavaScript*, focada na *performance* crua e na otimização de ficheiros estáticos para acesso instável. A resiliência da plataforma apoia-se em avançados recursos de armazenamento nativo.

3.3.1 Mecanismos de armazenamento

Tabela 3.2: Mecanismos de armazenamento no *browser* e respectivos propósitos.

Mecanismo	Dados guardados	Justificação	Expiração
<code>localStorage</code>	Preferências de visualização não sensíveis e estado colapsável da <i>Sidebar</i> .	Persiste entre sessões, garantindo a UX visual e reduzindo cliques desnecessários.	Manual ou na eventual limpeza da cache pelo navegador.
<code>IndexedDB</code>	Fila de operações (<i>Outbox</i>), nomeadamente medições ou tarefas marcadas offline.	Base de dados local transacional ideal para acumular pedidos falhados sem comprometer a <i>Thread</i> principal.	Purgado programaticamente pelo ficheiro <code>sync.js</code> após validação afirmativa da API.
Cache API	<i>Assets</i> imutáveis (HTML, CSS global, ícones SVG e <code>logo.png</code>).	Gerido pelo <code>service-worker.js</code> , permite servir a página com <i>latência-zero</i> sob falha total de rede.	Controlado pela versão do <i>Worker</i> ; renovado em <i>background</i> .

3.3.2 Decisão sobre o armazenamento do JWT

Decisão DA-01: Armazenamento do token JWT via *Cookies httpOnly*

Decisão. O token JWT gerado pelo *endpoint* de *Login* não é entregue ao cliente no *body* da resposta HTTP para ser guardado em `localStorage`. Em vez disso, é definido pelo servidor diretamente num *cookie* `httpOnly` com as *flags* `Secure` e `SameSite`.

Justificação. A manipulação e extração são controladas pelo *middleware* personalizado `cookieParser.js` no lado do Node.js. Esta escolha neutraliza ataques diretos de *Cross-Site Scripting* (XSS), visto que o próprio motor JavaScript do cliente (*browser*) está vedado de ler a propriedade `document.cookie` contendo a chave.

3.3.3 Estratégia de sincronização

Decisão DA-02: Retenção Offline e Sincronização Sequencial

Decisão. A aplicação adota um padrão *Offline-First* estrito, retendo operações localmente no *IndexedDB* sempre que houver falha de rede, retransmitindo-as de forma autônoma assim que a ligação for restabelecida.

Justificação. A operação em modo degradado apoia-se no módulo `shared/js/storage/sync.js` em conjunto com o `service-worker.js`. Quando um *Técnico* tenta submeter uma medição ambiental sem conectividade (*fetch* falha), o módulo capta o erro, embala o *payload* e insere-o na *Object Store* do *IndexedDB*. Assim que o navegador recupera o evento *online*, um despoletador retransmite a fila de forma sequencial. O projeto baseia-se num modelo isento de colisões para as *MedicoesAmbientais*, dado tratar-se de uma série temporal garantindo consistência por *append*.

3.4 Camada de servidor (API)

3.4.1 Cadeia de *middleware* e Rotas

O tráfego de entrada submete-se a um "túnel" de defesa e validação implementado por rotinas de *middleware* estritas e organizadas sequencialmente em `index.js`:

1. **CORS e Segurança:** Adoção do `cors.js` e do `securityHeaders.js` (bloqueio de *X-Frame-Options*, mitigação *MIME-Sniffing*).
2. **Rate Limiter:** Interceção precoce via `rateLimiter.js` para frustrar picos de requisições maliciosas.
3. **Sanitização e Parseamento:** `cookieParser.js` lê o JWT, seguido do `sanitize.js` que limpa caracteres de *MongoDB Injection* (ex: `$` e `.`) no *body*.
4. **Autenticação e Autorização:** `auth.js` converte e verifica o JWT. Consecutivamente, `requireRole.js` confronta as necessidades da rota com a *Role* decodificada no token.
5. **Controllers da Aplicação:** Acesso à lógica de negócio (*alertas*, *lotesCultivo*, *tarefas*, etc.).
6. **Gestão Centralizada de Erros:** `errorHandler.js` uniformiza as respostas HTTP de falha, assegurando o cumprimento da especificação documentada no `openapi.yaml`.

Decisão DA-03: Tratamento Centralizado de Erros (*Error Handling*)

Decisão. A gestão de exceções e erros da API não é feita de forma isolada e redundante dentro de cada controlador. Em vez disso, todas as exceções são canalizadas para um *middleware* global (`errorHandler.js`) posicionado no final da cadeia de execução do servidor Express.

Justificação. Esta centralização garante que qualquer falha (desde erros de validação do Mongoose a falhas inesperadas de rede) é formatada uniformemente. Assegura que o servidor nunca expõe a *stack trace* sensível em ambiente de produção e que devolve sempre uma resposta HTTP padronizada (com as propriedades `success`, `message` e `details`), mantendo total coerência com o contrato de interface definido no `openapi.yaml`.

3.4.2 Motor de regras IoT

Decisão DA-04: Avaliação por Motor de Regras e Emulação IoT

Decisão. Toda a avaliação estatística contra limites estabelecidos — desde regras simples a regras compostas de anomalia simultânea (ex: humidade alta e temperatura extrema) — corre no servidor. Adicionalmente, caso o lote esteja configurado para modo automático, a API emula os *payloads* de hardware IoT e cria instantaneamente tarefas corretivas executadas.

Justificação. O cliente web não é o ambiente apropriado para albergar lógicas agrícolas críticas. Concentrar o motor de regras (*Rules Engine*) no Node.js previne *bypasses* das regras de negócio. Se o código JavaScript do lado do cliente falhar ou for adulterado, a base de dados não ingere dados espúrios e garante a orquestração segura dos componentes físicos da estufa.

3.5 Modelo de dados

A adoção de MongoDB como repositório primário orientou a estrutura de dados a documentos JSON *schema-enforced*. A Figura 3.2 descreve conceptualmente as ligações do sistema.

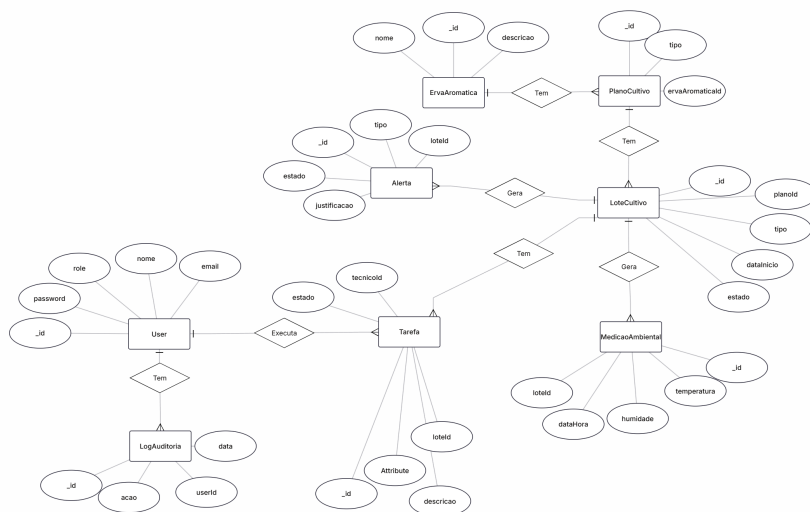


Figura 3.2: Modelo conceitual e associações na base de dados NoSQL.

Decisão DA-05: Padrão *Referencing* vs *Embedding* no MongoDB

Decisão. Adotou-se o modelo de **Referência Normalizada** em detrimento do alojamento incorporado (ex: o **LoteCultivo** contém apenas o **ObjectId** do seu **PlanoCultivo**).

Justificação. Um único Plano pode estar associado a centenas de Lotes distintos. O *embedding* do objeto plano causaria redundância gigantesca e uma anomalia de atualização absurda se houvesse necessidade de reajustar um limite de temperatura. As medições, altamente frequentes, mantêm igualmente o **ObjectId** do Lote correspondente, favorecendo uma inserção $O(1)$ e *queries* eficientes indexadas.

3.6 Segurança

As decisões complementares que consolidam o perfil de resiliência e a integridade da plataforma GREENHERB encontram-se estruturadas abaixo:

- **Trilha de Auditoria (*Audit Logging*):** Entidades críticas forçam injeção paralela de um documento na coleção **LogAuditoria**, fixando metadados fundamentais: o utilizador logado, o *timestamp*, o IP de origem e o recurso manipulado.

Decisão DA-06: Sanitização Estrita *Pre-Controller*

Decisão. Implementação de um *middleware* de intercepção precoce (`sanitize.js`) a nível global na aplicação Express, garantindo proteção antes de a requisição alcançar a camada dos controladores.

Justificação. A sanitização rigorosa limpa recursivamente o objeto `req.body` contra caracteres especiais (como \$ e .) típicos de injeção de comandos NoSQL, garantindo que o servidor rejeita ameaças de escalonamento de privilégios antes da execução transacional.

Decisão DA-07: Persistência do *Rate Limiting* no MongoDB

Decisão. A limitação de tentativas de acesso (*Rate Limiting*) não é gerida na memória volátil (RAM) do servidor Node.js, mas sim persistida ativamente na coleção `RateLimit` da base de dados MongoDB.

Justificação. Se as métricas de tráfego residissem apenas em memória, qualquer reinício do servidor (intencional ou provocado por um ataque de negação de serviço) faria o *reset* aos contadores, permitindo o retomar imediato de ataques de força bruta. Ao ancorar estes limites na base de dados, a plataforma retém a memória de ataques bloqueados de forma permanente, garantindo proteção ininterrupta.

Decisão DA-08: Derivação de Chaves Criptográficas com Argon2

Decisão. Para o armazenamento de palavras-passe na coleção de Utilizadores, rejeitou-se o uso de algoritmos clássicos de *hashing* e optou-se pela implementação do algoritmo **Argon2**.

Justificação. O Argon2 é o algoritmo vencedor da *Password Hashing Competition* e é o standard atual recomendado em cibersegurança. O Argon2 é *memory-hard*, o que significa que exige um elevado custo de memória RAM para ser computado. Esta decisão arquitetural protege a base de dados contra ataques de dicionário realizados por *clusters* de placas gráficas (GPUs), conferindo robustez máxima em caso de fuga de dados.

3.7 Resumo das decisões

A tabela abaixo compila as oito determinações arquiteturais fulcrais analisadas.

Tabela 3.3: Síntese das decisões de arquitetura e segurança.

ID	Decisão	Secção
DA-01	Acondicionamento e blindagem do JWT em <i>cookies httpOnly</i> .	3.3.2
DA-02	Retenção offline no <i>IndexedDB</i> e retransmissão sequencial por <i>Sync</i> .	3.3.3
DA-03	Tratamento centralizado de erros em <i>middleware</i> global da API.	3.4
DA-04	Avaliação IoT por Motor de Regras e injeção automática de ações.	3.4.2
DA-05	Prevalência de Padrão Mongoose <i>Referencing</i> para escalabilidade $O(1)$.	3.5
DA-06	Isolamento de sanitização estrita <i>pre-controller</i> (NoSQL Injection).	3.6
DA-07	Persistência de registos de limite de acessos (<i>Rate Limit</i>) em MongoDB.	3.6
DA-08	Derivação de chaves criptográficas avançada com proteção de memória (Argon2).	3.6

4

Conclusão

Este capítulo encerra o relatório do projeto GREENHERB, apresentando um balanço global sobre o cumprimento dos objetivos estipulados. Adicionalmente, reflete de forma crítica sobre as decisões técnicas adotadas, os obstáculos superados e as lições assimiladas pela equipa, propondo ainda um roteiro de melhorias para iterações vindouras.

4.1 Síntese do trabalho realizado

O projeto resultou na entrega de uma aplicação web *full-stack* funcional e arquitetonicamente robusta para a gestão de estufas agrícolas. No que concerne à cobertura dos Requisitos Funcionais, a esmagadora maioria dos requisitos classificados como obrigatórios (**M**ust-have) foi implementada com sucesso. Destacam-se o desenvolvimento do motor de autenticação e autorização por papéis, o registo integral do ciclo de vida de planos e lotes de cultivo, o sistema de interceção e geração de alertas, bem como as funcionalidades de exportação analítica de relatórios em formato CSV.

A nível de infraestrutura (*frontend* e *backend*), o projeto cumpre integralmente os requisitos não funcionais. Foi estabelecido um *middleware* de segurança assertivo (com *rate limiting* e sanitização) e uma arquitetura baseada em *Service Workers* e *IndexedDB* que garante a resiliência da aplicação (*Offline-First*). A total automatização proativa de componentes (emulação de rotinas IoT para hardware de rega e climatização) foi igualmente incluída na camada de serviços, dotando o backend de uma inteligência notável na gestão de conflitos ambientais.

4.2 Avaliação dos critérios de aceitação

A Tabela 4.1 confronta o trabalho realizado com os critérios de aceitação definidos no plano inicial.

Tabela 4.1: Verificação dos critérios de aceitação do projeto.

ID	Critério	Estado e Evidência
CA-01	Um utilizador autenticado acede apenas às operações permitidas pelo seu perfil.	✓ (Via <code>requireRole.js</code> e proteção de pastas estáticas).
CA-02	O sistema suporta os três tipos de plano de cultivo e a sua associação a lotes.	✓ (CRUD completo e cruzamento em <code>lotesCultivo.js</code>).
CA-03	O registo de medições ambientais pode originar alertas automáticos.	✓ (Motor de regras incorporado no <code>medicoesService.js</code>).
CA-04	O sistema demonstra comportamento distinto em modo <i>Manual</i> e <i>Automático</i> .	✓ (Em modo auto, a API gera <i>payloads</i> de hardware e executa as tarefas instantaneamente).
CA-05	É possível consultar histórico, exportar relatórios e observar rastreabilidade.	✓ (A rastreabilidade via <code>LogAuditoria</code> opera a 100%; exportação CSV implementada via <code>csvUtils.js</code>).
CA-06	A arquitetura de armazenamento no <i>browser</i> está implementada e coerente.	✓ (<code>sync.js</code> , <code>Service Worker</code> e <code>Cache API</code> operacionais).
CA-07	A implementation está alinhada com a especificação OpenAPI.	✓ (Documentação redigida no ficheiro <code>openapi.yaml</code>).

Legenda: ✓ (cumprido), ~ (parcialmente cumprido), × (não cumprido).

4.3 Reflexão crítica

4.3.1 O que correu bem

A principal vitória técnica da equipa residiu na arquitetura adotada para a API. A decisão de isolar responsabilidades em pequenos *middlewares* modulares — como o `sanitize.js`, `rateLimiter.js` e `cookieParser.js` — permitiu manter os controladores incrivelmente limpos e focados unicamente na lógica de negócio. Outro ponto forte foi a estratégia adotada para o armazenamento dos JWT. Recorrer a *cookies httpOnly* em vez de injetar os *tokens* via `localStorage` demonstrou uma preocupação proativa com a segurança, mitigando de forma muito eficaz possíveis ataques de XSS. A integração bem-sucedida do `swagger-ui-express` para servir a documentação OpenAPI e a sincronização *offline* nativa com *Vanilla JS* provaram que a equipa domina as ferramentas modernas de *backend* e *frontend*.

4.3.2 O que faríamos diferente

Em retrospectiva, a dependência exclusiva de *Vanilla JavaScript* para a manipulação intensiva do *Document Object Model* (DOM) tornou-se gradualmente num desafio. Na secção da *Dashboard*, gerir em simultâneo a renderização de lotes, alertas em tempo



Especificação OpenAPI

A especificação completa da API está disponível no ficheiro `openapi.yaml` no repositório do projeto. A título ilustrativo, apresenta-se a seguir o excerto correspondente ao *endpoint* de autenticação, que reflete a adoção de boas práticas na reutilização de componentes (esquemas) e a decisão arquitetural de não expor o token no corpo da resposta, utilizando em vez disso um *cookie httpOnly* (Decisão DA-01).

```
paths:
  /auth/login:
    post:
      tags:
        - Auth
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: "#/components/schemas/UserLogin"
      responses:
        "200":
          description: OK
          headers:
            Set-Cookie:
              schema:
                type: string
                description: Cookies access e refresh
          content:
            application/json:
              schema:
                $ref: "#/components/schemas/SuccessResponse"
        "401":
          description: Unauthorized
```

Listing A.1: Excerto de `openapi.yaml` — Rota de Autenticação.

B

Dados de demonstração

Para efeitos de validação e testes *end-to-end* (E2E) durante a apresentação final, o código-fonte faz-se acompanhar de um ficheiro `ervas_demo.csv` criado especificamente para validar a importação de dados, suportado por um conjunto de dados previamente preparados no MongoDB. O ecossistema de demonstração inclui:

- **9** espécies de ervas aromáticas (e.g., Manjerição, Hortelã, Tomilho, Alecrim, Oregãos) prontas a importar através do ficheiro `ervas_demo.csv`.
- **3** utilizadores pré-configurados (abrangendo os papéis obrigatórios: um *Administrador*, um *Responsável* e um *Técnico*).
- **4** planos de cultivo com diferentes tipologias (*regular*, *emergência* e *pontual*) e limites ambientais distintos.
- **8** lotes de cultivo em diferentes fases do ciclo de vida (*ativo*, *concluído* e *comprometido*).
- Histórico dinâmico composto por mais de **50** medições ambientais simuladas e **12** alertas despoletados (entre *Avisos* e estados *Críticos*) associados aos lotes ativos e ao motor de regras IoT.



Distribuição do trabalho

O desenvolvimento do projeto GREENHERB seguiu uma abordagem incremental dividida por *sprints*, onde as responsabilidades principais foram distribuídas pelos elementos do grupo para assegurar a cobertura total do ecossistema *full-stack*. A Tabela C.1 detalha a contribuição individual e as respectivas frentes de trabalho, refletindo o histórico de versionamento do repositório.

Tabela C.1: Contribuição de cada elemento da equipa.

Elemento	Contribuições principais	%
Letizia Coelli	Sprint 1: Conceção e desenvolvimento inicial do <i>Frontend</i> da aplicação utilizando <i>Vanilla JavaScript</i> ; estruturação das páginas estáticas de autenticação (login , register) e estilização dos componentes visuais base.	33.3%
Bruno Salvador	Sprints 2 e 4: Implementação da infraestrutura do servidor Node.js/Express e modelação em Mongoose (Sprint 2); Desenvolvimento do Motor de Regras IoT, arquitetura de retenção <i>offline</i> (IndexedDB/Sync) e refatorização de segurança na reta final (Sprint 4).	33.3%
Júlia Santos	Sprints 3 e 4: Conclusão do fluxo de autenticação e gestão de sessão; desenvolvimento das lógicas de importação e exportação de ficheiros (CSV/Excel); consolidação da especificação OpenAPI e redação integral do presente relatório técnico.	33.3%
Total		100%